BÁO CÁO THỰC HÀNH MINI PROJECT 03

MULTI-TIER ARCHITECTURE & TROUBLESHOOTING

Người báo cáo: Liamz07

1. Giới thiệu:

- **Mục tiêu:** Xây dựng, vận hành một Hệ thống ứng dụng mạng đa tầng (Multi-tier Network Application) trên môi trường Ubuntu Server, đồng thời chủ động giả lập sự cố (Fault Injection) để xây dựng quy trình chẩn đoán và khắc phục lỗi mạng tự động/hệ thống (Troubleshooting Methodology).
- **Sơ đồ luồng dữ liệu mạng:** Client (curl / Browser) → Nginx (Reverse Proxy) → Node.js App (PM2) → PostgreSQL (Port 5432)
- **Giải thích:**

  Tầng Web / Reverse Proxy (Nginx): Đóng vai trò làm điểm tiếp nhận duy nhất (Entry Point) ở cổng 80, thực hiện cân bằng tải và chuyển tiếp kết nối vào bên trong.

  Tầng Application (Node.js & PM2): Xử lý logic dịch vụ mạng ở cổng 3000, duy trì trạng thái hoạt động liên tục thông qua công cụ quản lý tiến trình PM2.

  Tầng Database (PostgreSQL): Lưu trữ dữ liệu cấu trúc dịch vụ ở tầng dưới cùng.

2. Triển khai và vận hành hệ thống nền tảng:

- **Cấu hình Nginx Reverse Proxy:** Tệp cấu hình tại /etc/nginx/sites-available/default được thiết lập để lắng nghe cổng 80 và proxy dữ liệu sang socket nội bộ.

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

- **Khởi tạo dịch vụ Node.js với PM2:** Mã nguồn ứng dụng mạng HTTP (app.js).

```
const http = require('http');
const hostname = '0.0.0.0';
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello from Node.js on Ubuntu Server!\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

Tiến trình được quản lý và đảm bảo tính sẵn sàng trong PM2: pm2 start app.js --name "web-app"

3. Nhật ký giả lập lỗi & Quy trình chẩn đoán (Fault Injection & Remediation Log):

Để hoàn thiện năng lực tự debug hệ thống mạng, 6 kịch bản sự cố đã được chủ động tạo ra và tài liệu hóa chi tiết.

KỊCH BẢN 1: LỖI 502 BAD GATEWAY (BACKEND SERVICE DOWN)

- **Nguyên nhân:** Tiến trình ứng dụng Node.js dừng hoạt động, Nginx không thể mở kết nối TCP tới cổng 3000.
- **Thao tác giả lập:** pm2 stop web-app
- **Kiểm tra & Dấu hiệu:**
  - *Chạy curl -I http://localhost → Trả về HTTP/1.1 502 Bad Gateway.*
  - *Log Nginx (/var/log/nginx/error.log) ghi nhận: connect() failed (111: Connection refused) while connecting to upstream.*
- **Khắc phục:** Khởi động lại dịch vụ pm2 restart web-app.

KỊCH BẢN 2: LỖI 503 SERVICE UNAVAILABLE (BẢO TRÌ/QUÁ TẢI)

- **Nguyên nhân:** Nginx chủ động cắt kết nối hoặc báo dịch vụ tạm thời không khả dụng.
- **Thao tác giả lập:** Thêm dòng return 503; (tạm thời sửa các dòng khác thành comment) vào khối location / trong cấu hình Nginx và chạy sudo systemctl reload nginx.
- **Kiểm tra & Dấu hiệu:** curl -I http://localhost trả về ngay lập tức mã HTTP/1.1 503 Service Unavailable.
- **Khắc phục:** Bỏ dòng return 503; và nạp lại cấu hình Nginx.

KỊCH BẢN 3: LỖI CONNECTION REFUSED (SAI CỔNG CẤU HÌNH UPSTREAM)

- **Nguyên nhân:** Nginx cấu hình trỏ vào một cổng không có tiến trình nào lắng nghe.
- **Thao tác giả lập:** Sửa proxy_pass http://localhost:3000; thành proxy_pass http://localhost:4000; (cổng 4000 đang trống).
- **Kiểm tra & Dấu hiệu:** Log Nginx báo lỗi ngắt kết nối Connection refused.
- **Khắc phục:** Sửa lại đúng thông số cổng 3000 trong file cấu hình Nginx và thực hiện sudo nginx -t && sudo systemctl reload nginx.

KỊCH BẢN 4: LỖI 504 GATEWAY TIME-OUT (XỬ LÝ BACKEND BỊ TRỄ)

- **Nguyên nhân:** Ứng dụng Node.js xử lý yêu cầu quá lâu, vượt ngưỡng thời gian chờ proxy_read_timeout của Nginx.
- **Thao tác giả lập:**
  - *Thêm proxy_read_timeout 1s; trong Nginx.*
  - *Thêm hàm setTimeout hoãn phản hồi 3 giây trong app.js và restart PM2.*
- **Kiểm tra & Dấu hiệu:** Sau 1 giây chờ, curl -I http://localhost trả về HTTP/1.1 504 Gateway Time-out.
- **Khắc phục:** Tối ưu hóa code backend hoặc điều chỉnh tăng thời hạn chờ proxy_read_timeout.

KỊCH BẢN 5: LỖI DNS RESOLUTION FAILURE (LỖI PHÂN GIẢI TÊN MIỀN)

- **Nguyên nhân:** Máy khách không thể tìm thấy địa chỉ IP tương ứng với tên miền truy cập.
- **Thao tác giả lập:** Thực hiện truy vấn tới một tên miền chưa đăng ký/không tồn tại: curl -I http://ten-mien-gia-lap.local
- **Kiểm tra & Dấu hiệu:** Terminal thông báo Could not resolve host: ten-mien-gia-lap.local.
- **Khắc phục:** Kiểm tra lại file cấu hình /etc/hosts hoặc bản ghi hệ thống DNS.

KỊCH BẢN 6: LỖI PORT BLOCKED / INTERNAL ISOLATION (CÔ LẬP CỔNG MẠNG)

- **Nguyên nhân:** Lớp mạng bị cô lập hoặc cổng truyền tải nội bộ bị chặn bởi chính sách bảo mật.
- **Phân tích kỹ thuật:** Trên Linux, các kết nối localhost chạy qua giao diện ảo Loopback (lo) được UFW cho phép mặc định. Khi giả lập chặn cổng nội bộ, ta nhận diện sự cố thông qua việc kiểm tra trạng thái socket bằng ss -tulpn và phân tích traffic mạng.
- **Khắc phục:** Đảm bảo các quy tắc định tuyến mạng nội bộ thông suốt và mở đúng các cổng giao tiếp dịch vụ (80, 3000, 5432).

4. Bộ công cụ chẩn đoán hệ thống đã sử dụng:

- **curl -I**: Kiểm tra các HTTP Response Header & Status Code.
- **pm2 status / logs / restart**: Quản lý và theo dõi sức khỏe tiến trình Node.js.
- **sudo nginx -t & sudo systemctl reload nginx**: Kiểm tra cú pháp và áp dụng thay đổi cấu hình Nginx an toàn.
- **sudo tail -f /var/log/nginx/error.log**: Theo dõi nhật ký lỗi Nginx theo thời gian thực.

- **ss -tulpn**: Kiểm tra danh sách các cổng mạng và tiến trình đang lắng nghe (Listening ports).

5. Kết luận:

- Dự án Mini Project 03 đã hoàn thiện trọn vẹn cả hai mục tiêu:

  Xây dựng thành công hạ tầng ứng dụng mạng đa tầng chuẩn mực (Nginx - Node.js - PostgreSQL).

  Tối ưu hóa kỹ năng "Auto-debug / Troubleshooting" dựa trên tư duy phân tích hệ thống: Quan sát triệu chứng → Tra cứu Log → Khoanh vùng nguyên nhân gốc rễ (Root Cause) → Khôi phục dịch vụ.

HẾT!